Problem Set 2

Xènia Alemany, Michael Fehl

2026-03-08

Table of contents

1. Community Detection **2**

(a) Node degrees and total edges . . . 2

(b) Modularity . . . 3

(c) Maximum-likelihood SBM parameters . . . 4

(d) Log-likelihood . . . 5

(e) Preferred partition . . . 5

(f) Discussion . . . 5

(g) Degree-corrected SBM . . . 6

2. Implementing the Locally Greedy Heuristic **6**

(a) Core Logic: `makeAMove()` . . . 6

(b) Phase Logic: `runOnePhase()` . . . 9

(c) Full Algorithm: `fitSBM()` . . . 13

(d) Zachary Karate Club . . . 16

(e) Extra Credit . . . 20

3. Node and Edge Predictions **20**

(a) Missing link prediction: Jaccard and Adar-Adamic . . . 20

(b) Missing attribute prediction: local smoothing . . . 21

4. Node2Vec Edge Prediction **22**

Imports . . . 22

(a) Generate Erdos-Renyi graph . . . 22

(b) Remove edges . . . 23

i. How many edges are in the original graph, and how many will you remove? . . . 23

ii. What strategy will you use to randomly select the edges to be removed? . . . 23

iii. How will you store or mark these removed edges for later evaluation? . . . 24

iv. What challenges might arise regarding the connectivity of the graph, and how would you address them? . . . 24
(c) Node2Vec embeddings . . . 25
i. What are the key parameters for your chosen embedding algorithm? . . . 25
ii. How does each parameter influence the quality and efficiency of the embeddings? . . . 25
iii. Can you visually represent the nodes in vector space? . . . 26
(d) Cosine similarity scoring . . . 27
i. How is cosine similarity defined mathematically? . . . 27
ii. Why might cosine similarity be an appropriate measure for scoring missing edges? . . . 29
iii. Discuss any potential benefits or drawbacks of using cosine similarity compared to other operators . . . 30
(e) ROC curve and AUC . . . 30
i. What does the ROC curve represent in the context of link prediction? . . . 30
ii. How do you interpret the AUC value with respect to the quality of your predictions? . . . 30
iii. What does an AUC value of 0.5 versus a value close to 1.0 indicate about your model's performance? . . . 31
(f) Analysis and discussion . . . 32
i. How effective was cosine similarity as a scoring method for predicting missing links? . . . 32
ii. How might different embedding parameters or alternative scoring methods impact the results? . . . 32
iii. What are the potential limitations of your current methodology, and what improvements would you suggest? . . . 34

1. Community Detection

Consider the undirected network G with nodes $1, \dots, 8$ and 11 edges given by: $E = \{(1,2),(1,3),(1,4),(1,5),(1,6),(1,7),(2,4),$

Candidate Partitions:

- **Partition A (structural communities):** $\{1,8\},\{2,3,4\},\{5,6,7\}$
- **Partition B (degree grouping):** $\{1\},\{2,3,4,5,6,7,8\}$

(a) Node degrees and total edges

Compute the degree of each node and the total number of edges m.

The degree of each node k_i is the number of edges incident to node i. We count the occurrences of each node in the edge list above:

Node (i)	Neighbors	Degree (k_i)
1	{2, 3, 4, 5, 6, 7}	**6**
2	{1, 4, 5}	**3**

Node (i)	Neighbors	Degree (k_i)
3	{1, 4}	**2**
4	{1, 2, 3}	**3**
5	{1, 2, 6, 7}	**4**
6	{1, 5}	**2**
7	{1, 5}	**2**
8	$\emptyset$	**0**

To find m, we count the unique pairs listed in the edge set E: 1. (1, 2) 2. (1, 3) 3. (1, 4) 4. (1, 5) 5. (1, 6) 6. (1, 7) 7. (2, 4) 8. (3, 4) 9. (2, 5) 10. (5, 6) 11. (5, 7)

Or alternatively following the class slides and using the information that the network is undirected:

$m = \frac{1}{2}\sum_{i=1}^{n} k_i$

$m = \frac{1}{2}(6+3+2+3+4+2+2+0) = 11$

The total number of edges is $m = 11$.

(b) Modularity

For each partition compute the modularity Q:

$$Q = \sum_c \left[\frac{m_c}{m} - \left(\frac{k_c}{2m}\right)^2\right]$$

where m_c denotes the number of edges inside community c and k_c the sum of degrees of nodes in c.

From a) we know that $m = 11$ and $2m = 22$

Community A

- Group c_1: $m_{c_1} = 0$ Because there are no internal edges between 1 and 8. $k_{c_1} = k_1 + k_8 = 6 + 0 = 6$ Contribution to Q: $\frac{0}{11} - (\frac{6}{22})^2 \approx -0.0744$
- Group c_2: $m_{c_2} = 2$ Because there are only two edges between the group nodes. $k_{c_2} = k_2 + k_3 + k_4 = 3 + 2 + 3 = 8$ Contribution to Q: $\frac{2}{11} - (\frac{8}{22})^2 \approx 0.0496$
- Group c_3: $m_{c_3} = 2$ Because there are only two edges between the group nodes. $k_{c_3} = k_5 + k_6 + k_7 = 4 + 2 + 2 = 8$ Contribution to Q: $\frac{2}{11} - (\frac{8}{22})^2 \approx 0.0496$

$Q \approx -0.0744 + 0.0496 + 0.0496 = 0.0248$

Community B

- Group c_1: $m_{c_1} = 0$ because there are no self-loops for node 1. $k_{c_1} = k_1 = 6$ Contribution to Q: $\frac{0}{11} - (\frac{6}{22})^2 \approx -0.0744$

Group c_2: $m_{c_2} = 5$ (Edges: (2,4), (3,4), (2,5), (5,6), (5,7)) $k_{c_2} = k_2 + k_3 + k_4 + k_5 + k_6 + k_7 + k_8 = 16$ Contribution to Q: $\frac{5}{11} - (\frac{16}{22})^2 \approx -0.0744$

$Q_B \approx -0.1488$

(c) Maximum-likelihood SBM parameters

For each partition compute the maximum-likelihood SBM parameters:

$$\tilde{M}_{uv} = \frac{E_{uv}}{N_{uv}}$$

where E_{uv} is the number of edges between blocks u and v and N_{uv} the number of possible such edges.

For undirected graphs without self-loops, if $u = v$, $N_{uu} = \frac{n_u(n_u-1)}{2}$; if $u \neq v$, $N_{uv} = n_u n_v$

Partition A (Structural Communities)

Blocks: $1 : \{1, 8\}$, $2 : \{2, 3, 4\}$, $3 : \{5, 6, 7\}$.

Block Sizes: $n_1 = 2, n_2 = 3, n_3 = 3$.

Block 1 ($u = 1$): $E_{11} = 0$ (No edge between 1 and 8). $N_{11} = \frac{2(1)}{2} = 1$. $\tilde{M}_{11} = \frac{0}{1} = 0$.

Block 2 ($u = 2$): $E_{22} = 2$ (Edges: (2,4), (3,4)). $N_{22} = \frac{3(2)}{2} = 3$. $\tilde{M}_{22} = \frac{2}{3} \approx 0.667$.

Block 3 ($u = 3$): $E_{33} = 2$ (Edges: (5,6), (5,7)). $N_{33} = \frac{3(2)}{2} = 3$. $\tilde{M}_{33} = \frac{2}{3} \approx 0.667$.

Between Blocks 1 & 2: $E_{12} = 3$ (Edges: (1,2), (1,3), (1,4)). $N_{12} = 2 \times 3 = 6$. $\tilde{M}_{12} = \frac{3}{6} = 0.5$.

Between Blocks 1 & 3: $E_{13} = 3$ (Edges: (1,5), (1,6), (1,7)). $N_{13} = 2 \times 3 = 6$. $\tilde{M}_{13} = \frac{3}{6} = 0.5$.

Between Blocks 2 & 3: $E_{23} = 1$ (Edge: (2,5)). $N_{23} = 3 \times 3 = 9$. $\tilde{M}_{23} = \frac{1}{9} \approx 0.111$.

Partition B (Degree Grouping)

Blocks: $1 : \{1\}$, $2 : \{2, 3, 4, 5, 6, 7, 8\}$.

Block Sizes: $n_1 = 1, n_2 = 7$.

Block 1 ($u = 1$):$E_{11} = 0.N_{11} = \frac{1(0)}{2} = 0$ (Not applicable/Undefined).$\tilde{M}_{11} = 0$.

Block 2 ($u = 2$):$E_{22} = 5$ (Edges: (2,4), (3,4), (2,5), (5,6), (5,7)).$N_{22} = \frac{7(6)}{2} = 21.\tilde{M}_{22} = \frac{5}{21} \approx 0.238$.

Between Blocks 1 & 2:$E_{12} = 6$ (Edges: (1,2), (1,3), (1,4), (1,5), (1,6), (1,7)).$N_{12} = 1 \times 7 = 7.\tilde{M}_{12} = \frac{6}{7} \approx 0.857$.

Final results:

Partition A

Blocks (u, v)	E_{uv}	N_{uv}	$\tilde{M}_{uv}$
(1,1)	0	1	0.000
(2,2)	2	3	0.667
(3,3)	2	3	0.667
(1,2)	3	6	0.500
(1,3)	3	6	0.500
(2,3)	1	9	0.111

Partition B

Blocks (u, v)	E_{uv}	N_{uv}	$\tilde{M}_{uv}$
(1,1)	0	0	0.000
(2,2)	5	21	0.238
(1,2)	6	7	0.857

(d) Log-likelihood

Using these estimates, compute the log-likelihood:

$$\ln L(G|z) = \sum_{u,v} \left[E_{uv} \ln \frac{E_{uv}}{N_{uv}} + (N_{uv} - E_{uv}) \ln \frac{N_{uv} - E_{uv}}{N_{uv}} \right]$$

Partition A

Block (1,1): $E_{11} = 0, N_{11} = 1$. - Log-likelihood contribution: $0 \ln(0) + (1-0)\ln(1/1) = 0$.

Block (2,2): $E_{22} = 2, N_{22} = 3$. - Contribution: $2\ln(2/3) + 1\ln(1/3) \approx -0.811 - 1.099 = -1.910$.

Block (3,3): $E_{33} = 2, N_{33} = 3$. - Contribution: $2\ln(2/3) + 1\ln(1/3) \approx -1.910$.

Block (1,2): $E_{12} = 3, N_{12} = 6$. - Contribution: $3\ln(3/6) + 3\ln(3/6) = 6\ln(0.5) \approx -4.159$.

Block (1,3): $E_{13} = 3, N_{13} = 6$. - Contribution: $3\ln(3/6) + 3\ln(3/6) = 6\ln(0.5) \approx -4.159$.

Block (2,3): $E_{23} = 1, N_{23} = 9$. - Contribution: $1\ln(1/9) + 8\ln(8/9) \approx -2.197 - 0.942 = -3.139$.

Total $\ln L(G|z_A) \approx 0 - 1.910 - 1.910 - 4.159 - 4.159 - 3.139 = -15.277$

Block (1,1): $N_{11} = 0$. - No contribution.

Block (2,2): $E_{22} = 5, N_{22} = 21$. - Contribution: $5\ln(5/21) + 16\ln(16/21) \approx -7.175 - 4.346 = -11.521$.

Block (1,2): $E_{12} = 6, N_{12} = 7$. - Contribution: $6\ln(6/7) + 1\ln(1/7) \approx -0.925 - 1.946 = -2.871$.

Total $\ln L(G|z_B) \approx -11.521 - 2.871 = -14.392$

(e) Preferred partition

Which partition does modularity prefer? Which does SBM likelihood prefer?

According to modularity, which partition is preferred? According to the SBM likelihood, which partition is preferred?

According to modularity: **Partition A**

According to the SBM likelihood: **Partition B**

(f) Discussion

Explain carefully why the two criteria differ in this example. Your explanation should refer to: * The configuration-model null underlying modularity. * The homogeneity assumptions of the standard SBM. * The role of the high-degree hub.

Modularity compares the observed network to a configuration model null hypothesis, where the expected number of edges between nodes is determined by their degrees. Because node 1 has a very high degree ($k_1 = 6$), the null model "expects" it to be connected to almost everyone. When Partition B places node 1 in its own group, it creates a community with zero internal edges, even though the null model expects many; this results in a high penalty that drives the modularity score Q below zero.

The standard SBM assumes homogeneity within blocks, meaning all nodes in a group must have similar connectivity patterns (stochastic equivalence). Partition A violates this by grouping node 1 (a hub) with node 8 (an isolate). The SBM model finds Partition A less "likely" because it is forced to average the behavior of very different nodes, whereas Partition B isolates the hub, allowing for a much more precise statistical fit for the edges connecting to it. Node 1 acts as a central hub that bridges the structural communities. While modularity focuses on the "assortative" clusters (the triangles in groups {2,3,4} and {5,6,7}), the SBM is

more sensitive to the degree heterogeneity. The SBM prefers Partition B because isolating the high-degree hub into its own block provides the most accurate description of the global edge distribution, even if that group lacks internal edges.

(g) Degree-corrected SBM

Briefly explain how the degree-corrected SBM would change the analysis.

The degree-corrected SBM (DC-SBM) would significantly alter the analysis by relaxing the standard model's strict requirement that all nodes within a block share the same expected degree. In the standard SBM, nodes are considered stochastically equivalent, which forces the model to prefer Partition B simply to separate the high-degree hub from the low-degree nodes. The DC-SBM introduces an additional parameter for each node that accounts for its individual degree, decoupling the node's propensity to form edges from its community assignment.

By allowing for such degree heterogeneity, the DC-SBM would no longer penalize Partition A for grouping node 1 and node 8 together. Instead of being forced to isolate the hub to achieve a better statistical fit, the model could assign a high degree parameter to node 1 and a low one to node 8 while keeping them in the same structural block. This shift allows the likelihood calculation to focus on the underlying community structure rather than the sheer number of edges connected to node 1, likely leading the model to prefer the structural communities of Partition A over the degree-based grouping of Partition B.

---

2. Implementing the Locally Greedy Heuristic

Implement and apply the locally greedy heuristic for estimating the SBM.

(a) Core Logic: `makeAMove()`

Write a function `makeAMove()` that implements the core logic: * **Input:** Graph G, current partition z_t, number of groups c, and a binary "isFrozen" vector f. * **Task:** Loop over $n - t$ unfrozen nodes and $c - 1$ possible moves to track the best log-likelihood $\mathcal{L}_*$ and the corresponding best move (i, z_i). * **Application:** Test on the graph in Figure 1 with $c = 3$. Visualize the input graph and the graph after the best move, noting the log-likelihoods.

We first need to also create a function to compute the log-likelihood given certain parameters:

```
# Log-likelihood:
compute_log_likelihood <- function(G, z, c){
  L <- 0
  for (u in 1:c){
    for (v in u:c) {
      nodes_u <- which(z == u)
      nodes_v <- which(z == v)
      n_u <- length(nodes_u)
      n_v <- length(nodes_v)
      if (u == v) {
        if (n_u == 0 || n_v == 0) next
        N_uv <- n_u * (n_u - 1) / 2 # n_u choose 2
        E_uv <- sum(G[nodes_u, nodes_u]) / 2
      } else {
```

```
        N_uv <- n_u * n_v
        if (N_uv == 0) next
        E_uv <- sum(G[nodes_u, nodes_v])
      }
      M <- E_uv / N_uv
      term1 <- if (E_uv > 0) E_uv * log(M) else 0
      term2 <- if (N_uv - E_uv > 0) (N_uv - E_uv) * log(1 - M) else 0
      factor <- if (u == v) 1 else 2
      L <- L + factor * (term1 + term2)
    }
  }
  return(L)
}

# Create the function:

makeAMove <- function(G, z_t, c, f){
  unfrozen_nodes <- which(f == 0) # know which nodes are unfrozen
  best_L <- -Inf # give an initial value to compare. That way there is no need to compute anything extra
  best_node <- NULL # for the output
  best_group <- NULL # for the output

  for (i in unfrozen_nodes) {
    old_group <- z_t[i]
    for (possible_group in 1:c) {
      if (possible_group == old_group) next
      z_t[i] <- possible_group
      current_L <- compute_log_likelihood(G, z_t, c)
      #print(paste0(i, " ",possible_group," ",current_L))

      if (current_L > best_L) {
        best_L <- current_L
        best_node <- i
        best_group <- possible_group
      }
    }
    z_t[i] <- old_group # Put back the node into its group to continue with the comparison
  }
  return(list(L_star = best_L, move = c(best_node, best_group)))
}
```

Apply the function to figure 1:

```
G <- matrix(0L, 9, 9)
edges <- rbind(c(1,2), c(1,3), c(1,6), c(1,7), c(1,8), c(1,9),
               c(3,4), c(3,6), c(4,6), c(5,6))

for (e in 1:nrow(edges)) {
  G[edges[e,1], edges[e,2]] <- 1L
  G[edges[e,2], edges[e,1]] <- 1L   # symmetric - undirected graph
}

c <- 3
```

```r
frozen <- rep(0L,9)

frozen[2] <- 1

z_t <- sample(1:c,9,replace = T)

print(z_t)
```

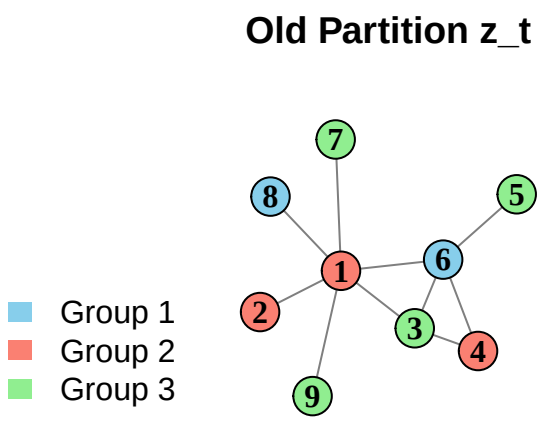
```
[1] 2 2 3 2 3 1 3 1 3
```

```r
output <- makeAMove(G, z_t, c, frozen)

new_z_t <- z_t

new_z_t[output$move[1]] <- output$move[2]

print(new_z_t)
```

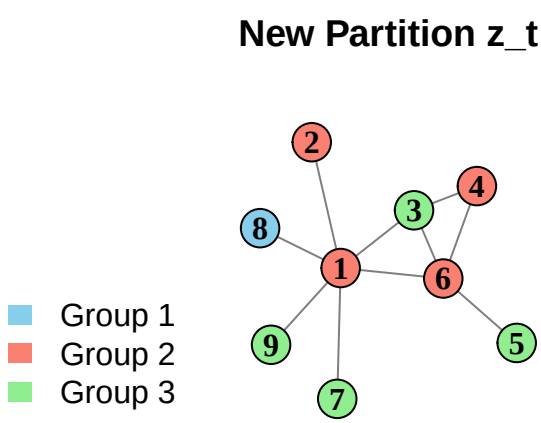
```
[1] 2 2 3 2 3 2 3 1 3
```

Plot the graph:

```r
library(igraph)
group_colors <- c("skyblue", "salmon", "lightgreen")
node_colors  <- group_colors[z_t]
g <- graph_from_adjacency_matrix(G, mode = "undirected")

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:9,
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "Old Partition z_t")

legend("bottomleft",
       legend = c("Group 1", "Group 2", "Group 3"),
       fill   = group_colors,
       border = NA,
       bty    = "n")
```




```
# new graph with corrected node

group_colors <- c("skyblue", "salmon", "lightgreen")
node_colors  <- group_colors[new_z_t]
g <- graph_from_adjacency_matrix(G, mode = "undirected")

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:9,
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "New Partition z_t")

legend("bottomleft",
       legend = c("Group 1", "Group 2", "Group 3"),
       fill   = group_colors,
       border = NA,
       bty    = "n")
```



(b) Phase Logic: `runOnePhase()`

Write a function `runOnePhase()` that runs a single phase until all nodes are frozen. * **Output:** Best partition z_*, log-likelihood $\mathcal{L}_*$, halting indicator h, and the list of log-likelihood values l_t for the $n+1$ partitions. *

Application: Visualize the input, the end-of-phase partition, and the evolution of l_t.

```
runOnePhase <- function(G, z_0, c, L_0 =NULL) {
  n <- length(z_0)
  z_t <- z_0
  f <- rep(0, n) # 0 = unfrozen

  history_z <- list(z_0)
  history_L <- c(compute_log_likelihood(G, z_0, c))

  for (t in 1:n) {
    res <- makeAMove(G, z_t, c, f)

    # Update partition and freeze node
    z_t[res$move[1]] <- res$move[2] # assign the move
    f[res$move[1]] <- 1 # freeze the node moved

    history_z[[t + 1]] <- z_t
    history_L[t + 1] <- res$L_star
  }

  best_idx <- which.max(history_L)
  z_star <- history_z[[best_idx]]
  L_star <- history_L[best_idx]

  # Halt if best partition in phase is not better than start
  baseline <- if (!is.null(L_0)) L_0 else history_L[1]
  halt <- L_star <= baseline

  return(list(z_star = z_star, L_star = L_star, halt = halt, l_t = history_L))
}
```

Apply the second function to the example graph:

```
G <- matrix(0L, 9, 9)
edges <- rbind(c(1,2), c(1,3), c(1,6), c(1,7), c(1,8), c(1,9),
               c(3,4), c(3,6), c(4,6), c(5,6))

for (e in 1:nrow(edges)) {
  G[edges[e,1], edges[e,2]] <- 1L
  G[edges[e,2], edges[e,1]] <- 1L   # symmetric - undirected graph
}

c <- 3

z_0 <- sample(1:c,9,replace = T)

print(z_0)
```

```
[1] 2 3 2 2 3 1 3 3 2
```

```
output <- runOnePhase(G, z_0, c)
output$z_star
```

```
[1] 1 3 2 2 2 2 3 3 3
```

```
output$L_star
```

```
[1] -9.364262
```

```
output$halt
```

```
[1] FALSE
```

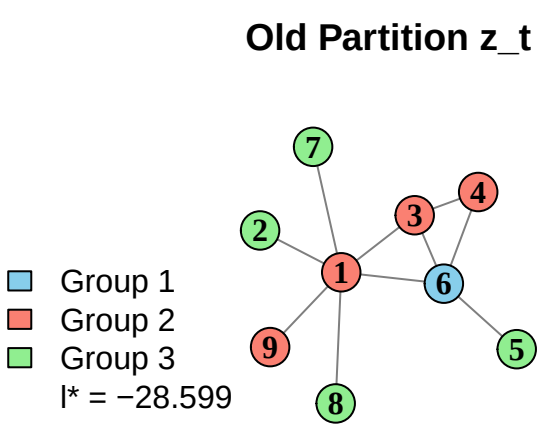
```
output$l_t
```

```
 [1] -28.598727 -20.638067 -18.361625 -16.171010  -9.364262 -13.460233
 [7] -16.336898 -18.600941 -24.341756 -25.763716
```

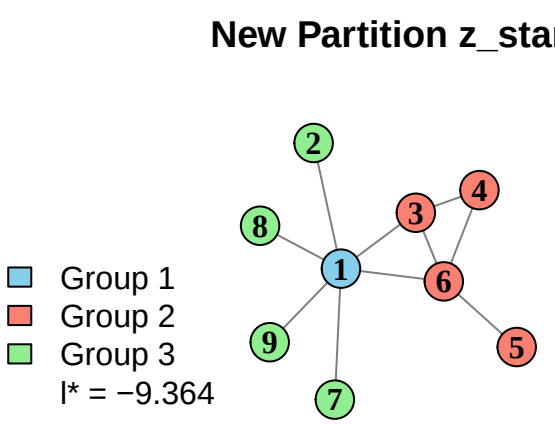
The first print corresponds to the original random z_0 allocation, followed by the optimal z_t the program found with its corresponding maximum log-likelihood associated.

Finally, a value True or False on whether the halting criteria was met or not plus all the log-likelihoods.

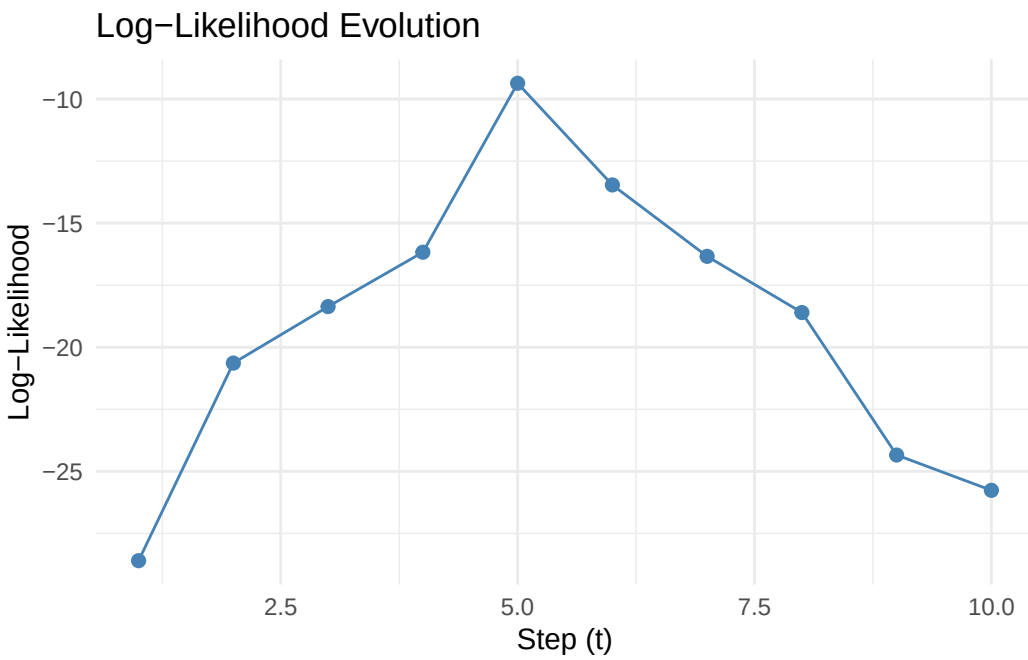
```
group_colors <- c("skyblue", "salmon", "lightgreen")
node_colors  <- group_colors[z_0]
g <- graph_from_adjacency_matrix(G, mode = "undirected")

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:9,
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "Old Partition z_t")

legend("bottomleft",
       legend = c("Group 1", "Group 2", "Group 3",
                  paste0(" * = ", round(output$l_t[1], 3))),
       fill   = c(group_colors, NA),
       border = c("black","black","black", NA),
       bty    = "n")
```



```r
group_colors <- c("skyblue", "salmon", "lightgreen")
node_colors  <- group_colors[output$z_star]
g <- graph_from_adjacency_matrix(G, mode = "undirected")

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:9,
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "New Partition z_star")

legend("bottomleft",
       legend = c("Group 1", "Group 2", "Group 3",
                  paste0(" * = ", round(output$L_star, 3))),
       fill   = c(group_colors, NA),
       border = c("black","black","black", NA),
       bty    = "n")
```




```r
library(ggplot2)

ggplot(data.frame(t = seq_along(output$l_t), ll = output$l_t), aes(x = t, y = ll)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue", size = 2) +
  labs(x = "Step (t)", y = "Log-Likelihood", title = "Log-Likelihood Evolution") +
  theme_minimal()
```



(c) Full Algorithm: `fitSBM()`

Write a function `fitSBM()` that runs the full heuristic (multiple phases). * **Input:** Graph G, number of groups c, and maximum phases T. * **Output:** Best partition z_*, log-likelihood $\mathcal{L}_*$, number of phases p_c, and full log-likelihood history. * **Application:** Apply to Figure 1 with $c = 3, T = 30$. Report the final vector and ω matrix. Comment on the partition quality and l_t behavior.

```
fitSBM <- function(G, c, T_max) {
  n <- nrow(G)
  # Random initial partition
  z_0 <- sample(1:c, n, replace = TRUE)
  L_0 <- compute_log_likelihood(G, z_0, c)

  all_L_history <- c()
  num_phases <- 0
  z_current <- z_0

  for (p in 1:T_max) {
    num_phases <- num_phases + 1
    res <- runOnePhase(G, z_current, c, L_0 = L_0)
    all_L_history <- c(all_L_history, res$l_t)

    if (res$halt) {
      #print('Halted')
      break
    } #break # Halt and return current
    #print('Not halted')
    z_current <- res$z_star #update when not halting
    L_0 <- res$L_star
  }

  return(list(z_star = res$z_star, L_star = res$L_star,
```

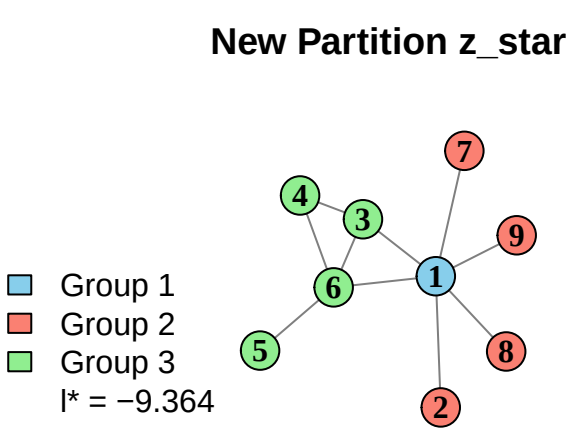
```
              phases = num_phases, history = all_L_history, z_0 = z_0))
}
```

Now again, we apply the code to the figure 1 example:

```
G <- matrix(0L, 9, 9)
edges <- rbind(c(1,2), c(1,3), c(1,6), c(1,7), c(1,8), c(1,9),
               c(3,4), c(3,6), c(4,6), c(5,6))

for (e in 1:nrow(edges)) {
  G[edges[e,1], edges[e,2]] <- 1L
  G[edges[e,2], edges[e,1]] <- 1L   # symmetric - undirected graph
}

c <- 3

output <- fitSBM(G, c, T_max = 30)
```

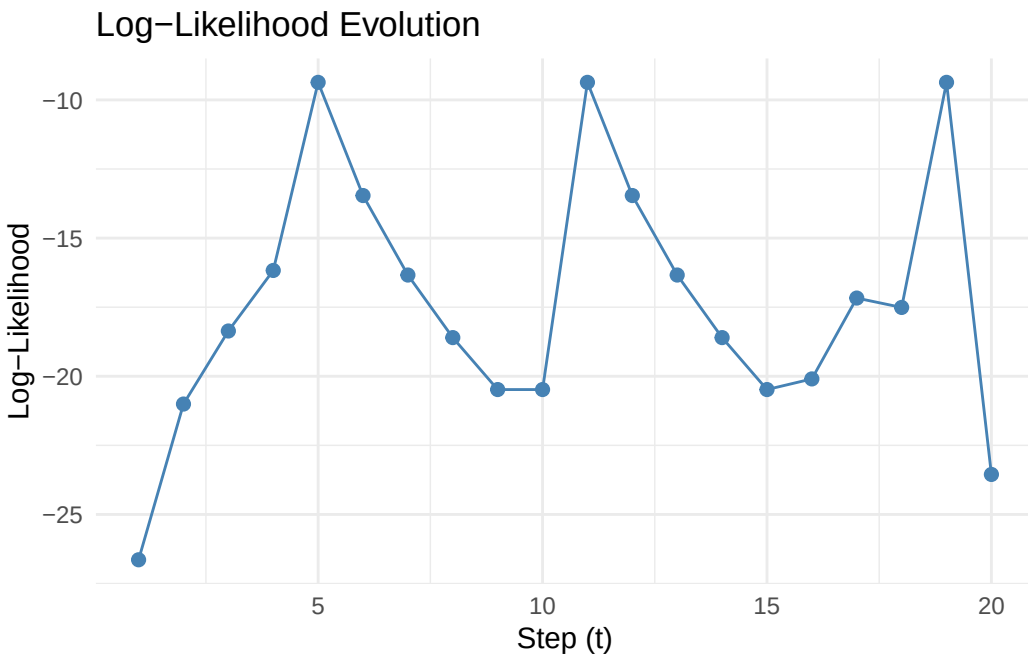
Graph with the final z_star and L_star:

```
group_colors <- c("skyblue", "salmon", "lightgreen")
node_colors  <- group_colors[output$z_star]
g <- graph_from_adjacency_matrix(G, mode = "undirected")

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:9,
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "New Partition z_star")

legend("bottomleft",
       legend = c("Group 1", "Group 2", "Group 3",
                  paste0(" * = ", round(output$L_star, 3))),
       fill   = c(group_colors, NA),
       border = c("black","black","black", NA),
       bty    = "n")
```



Evolution of the log-likelihood:

```
ggplot(data.frame(t = seq_along(output$history), ll = output$history), aes(x = t, y = ll)) +
  geom_line(color = "steelblue") +
  geom_point(color = "steelblue", size = 2) +
  labs(x = "Step (t)", y = "Log-Likelihood", title = "Log-Likelihood Evolution") +
  theme_minimal()
```




```
kappa <- table(output$z_star)
print(kappa)
```

```
1 2 3
1 4 4
```

```
compute_omega <- function(G, z, c) {
  omega <- matrix(0, c, c)
  for (r in 1:c) {
    for (s in r:c) {
      nodes_r <- which(z == r)
      nodes_s <- which(z == s)
      if (r == s) {
        N_rs <- length(nodes_r) * (length(nodes_r) - 1) / 2
        E_rs <- sum(G[nodes_r, nodes_r]) / 2
      } else {
        N_rs <- length(nodes_r) * length(nodes_s)
        E_rs <- sum(G[nodes_r, nodes_s])
      }
      omega[r, s] <- if (N_rs > 0) E_rs / N_rs else 0
      omega[s, r] <- omega[r, s]   # symmetric
    }
```

```r
  }
  return(omega)
}

omega <- compute_omega(G, output$z_star, c)
print(round(omega, 3))
```

```
     [,1] [,2]  [,3]
[1,]  0.0    1 0.500
[2,]  1.0    0 0.000
[3,]  0.5    0 0.667
```

Kappa vector represents the group sizes and w matrix is the estimated edge probabilities between groups. w_{rs} tells the estimated probability of an edge between a node in group r and a node in group s. High diagonal values mean dense within-group connections (i.e good community structure), while low off-diagonal values mean sparse between-group connections.

kappa

The algorithm assigned 4 nodes to group 1, 4 to group 2, and only 1 to group 3. This is somewhat unbalanced, and the singleton group 3 is a warning sign, a single node cannot form a meaningful community.

w matrix

The partition quality is poor. A well-detected community structure should show high on the diagonal and low values off it. Here we see the reverse, suggesting the algorithm got trapped in a local optimum due to the random initialization.

log evolution

The log-likelihood shows a volatile, non-monotone pattern across the ~20 steps recorded. Within each phase it rises to a local peak then drops sharply then this drop marks the start of a new phase where a fresh greedy search begins from the previous best partition. The peaks around t = 4, t = 11, and t = 19 correspond to the best partitions found at the end of each phase, which is consistent with the algorithm running approximately 2-3 phases before halting. Notably the peaks do not improve much across phases (all around -10), suggesting the algorithm converged quickly to the same local optimum regardless of the within-phase exploration.

(d) Zachary Karate Club

Apply `fitSBM()` repeatedly to the Zachary karate club network. Tinker with repetitions to find the optimal $c = 2$ partition. Comment on the number of runs required to reach the optimum.

```r
# Function to find the best SBM partition over multiple repetitions
findBestSBM <- function(G, c, repetitions, T_max) {
  best_overall_L <- -Inf
  best_overall_z <- NULL

  for (i in 1:repetitions) {
    # fitSBM() from previous part
    result <- fitSBM(G, c, T_max)
    print(result$L_star)

    if (result$L_star > best_overall_L) {
      best_overall_L <- result$L_star
      #print(result$L_star)
      best_overall_z <- result$z_star
```

```
    }
  }

  return(list(z_best = best_overall_z, L_best = best_overall_L, z_0 = result$z_0))
}

# Load the necessary libraries
library(igraph)     # Provides as_adjacency_matrix
library(igraphdata) # Provides the karate dataset

# Load the data
data(karate)

# Convert to adjacency matrix
adj_karate <- as_adjacency_matrix(karate, sparse = FALSE)

c <- 2

# Now you can run your optimization function
karate_results <- findBestSBM(adj_karate, c, repetitions = 10, T_max = 30)
```

```
[1] -226.8056
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
[1] -226.2021
```

```
table(karate_results$z_best)
```

```
 2
34
```

Apparently, when we run the code for c = 2, the algorithms optimal decision is to move all nodes into one single group. With only 2 groups the penalty for between-group edges is relatively weak compared to c = 3 or 4. We try to fix the issue but imposing to have balanced groups:

```
compute_log_likelihood <- function(G, z, c){
  L <- 0
  n <- length(z)
  group_sizes <- tabulate(z, nbins = c)

  # Each group must have at least n/(2*c) nodes
  min_size <- max(2, floor(n / (2 * c)))
  if (any(group_sizes < min_size)) return(-Inf)

  for (u in 1:c){
```

```
    for (v in u:c) {
      nodes_u <- which(z == u)
      nodes_v <- which(z == v)
      n_u <- length(nodes_u)
      n_v <- length(nodes_v)
      if (u == v) {
        N_uv <- n_u * (n_u - 1) / 2
        E_uv <- sum(G[nodes_u, nodes_u]) / 2
      } else {
        N_uv <- n_u * n_v
        if (N_uv == 0) next
        E_uv <- sum(G[nodes_u, nodes_v])
      }
      M <- E_uv / N_uv
      term1 <- if (E_uv > 0) E_uv * log(M) else 0
      term2 <- if (N_uv - E_uv > 0) (N_uv - E_uv) * log(1 - M) else 0
      factor <- if (u == v) 1 else 2
      L <- L + factor * (term1 + term2)
    }
  }
  return(L)
}
```

We rerun the code for karate only:

```
c <- 2

# Now you can run your optimization function
karate_results <- findBestSBM(adj_karate, c, repetitions = 10, T_max = 30)
```

```
[1] -239.7155
[1] -239.7155
[1] -248.4982
[1] -239.7155
[1] -239.7155
[1] -248.4982
[1] -239.7155
[1] -248.4982
[1] -248.4982
[1] -248.4982
```

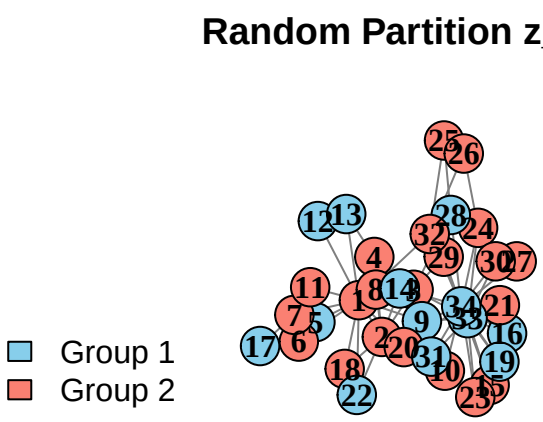
```
table(karate_results$z_best)
```

```
 1  2
16 18
```

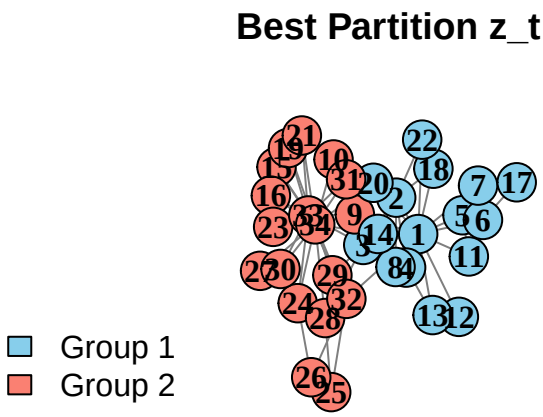
Now it created a balanced split

```
group_colors <- c("skyblue", "salmon")
node_colors  <- group_colors[karate_results$z_0]
g <- graph_from_adjacency_matrix(adj_karate, mode = "undirected")
```

```r
# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:length(karate_results$z_best),
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "Random Partition z_t")

legend("bottomleft",
       legend = c("Group 1", "Group 2"),
       fill   = c(group_colors),
       border = c("black","black"),
       bty    = "n")
```




```r
group_colors <- c("skyblue", "salmon")
node_colors  <- group_colors[karate_results$z_best]

# Plot
plot(g,
     vertex.color = node_colors,
     vertex.label = 1:length(karate_results$z_best),
     vertex.size  = 30,
     vertex.label.color = "black",
     vertex.label.font  = 2,
     edge.color   = "gray50",
     main         = "Best Partition z_t")

legend("bottomleft",
       legend = c("Group 1", "Group 2"),
       fill   = c(group_colors),
       border = c("black","black"),
       bty    = "n")
```



Now the split the algorithm is pretty close to the class slides one.

(e) Extra Credit

Apply your SBM algorithm to a real-world network ($n < 100$ recommended). Visualize G with z_* and comment on the structure (assortative, core-periphery, etc.).

---

3. Node and Edge Predictions

Consider the labeled graph provided in the assignment.

(a) Missing link prediction: Jaccard and Adar-Adamic

Imagine we know there is a missing link between node B and some other node. Using the Jaccard Index, tell me which link(s) is most likely to be missing. Would the answer change if we use the Adar-Adamic index?

The Jaccard coefficient captures the fraction of neighbors of either node that are neighbors of both nodes. More formally,

$$Jaccard(i,j) = \frac{\mid N(v_1) \cap N(v_2) \mid}{\mid N(v_1) \cup N(v_2) \mid}$$

The Adar-Adamic index similarly captures overlap by looking at the inverse degree of shared friends. More formally,

$$A(i,j) = \sum_{u \in N(i) \cap N(j)} \frac{1}{log \mid N(u) \mid}$$

Where $N(v_1)$ represents the neighbors of node i (edges connecting to other nodes), and $N(v_2)$ represents the same for some other non-neighboring node j. Looking at the graph provided, using the Jaccard index we see that for Node B it has $N(B) = \{H, C, D\} = 3$. For all other non-neighboring nodes, $\{A, E, G, F\}$, we find their number of neighbors as:

Node	Neighbors	Number
A	$\{E, F, G, H\}$	4
E	$\{A, H\}$	3
G	$\{A\}$	1
F	$\{A\}$	1

Thus, we can look at simply the intersection and union of these nodes for each $B - v_2$ pair:

$$B{-}A = \frac{\mid N(B) \cap N(A) \mid}{\mid N(B) \cup N(A) \mid} = \frac{1}{6} B{-}E = \frac{\mid N(B) \cap N(E) \mid}{\mid N(B) \cup N(E) \mid} = \frac{1}{4} B{-}F = \frac{\mid N(B) \cap N(F) \mid}{\mid N(B) \cup N(F) \mid} = \frac{0}{4} B{-}G = \frac{\mid N(B) \cap N(G) \mid}{\mid N(B) \cup N(G) \mid} = \frac{0}{4}$$

Since $\frac{1}{4} > \frac{1}{6} > 0$, the Jaccard index predicts $B - E$ as the missing edge link.

For the Adar-Adamic index, we can simply plug into the formula defined above. Since both connections $B - A, B - E$ share one common neighbor, $\{H\}$, we find the index score based on the neighbors of node H:

$$A(B, A) = \sum_{u \in N(B) \cap N(A)} \frac{1}{log \mid N(H) \mid} = \frac{1}{log \mid 3 \mid} \approx 0.910 A(B, E) = \sum_{u \in N(B) \cap N(E)} \frac{1}{log \mid N(H) \mid} = \frac{1}{log \mid 3 \mid} \approx 0.910$$

Thus, the Adar-Adamic index gives us a tie between a missing link for either $B - A$ or $B - E$. Thus, the answer should not change if we select based on both metrics, which would give us a preference to impute a missing link as $B - E$.

(b) Missing attribute prediction: local smoothing

Imagine that a binary node attribute (for instance, gender) is incomplete. Predict the missing attributes (i.e. the white nodes) using the local smoothing algorithm we saw in class.

The local smoothing algorithm predicts a missing binary attribute as the **mode** (we would use mean as a measure for continuous values) among a node's neighbors with known attributes. More formally:

$$x_i^* = \text{mode}\left(\{x^\circ_{N(i)}\} - \emptyset\right) \quad \text{if } x \text{ is categorical}$$

Where $x^\circ_{N(i)}$ represents the set of known attribute values of node i's neighbors. Predictions for all unlabeled nodes are made in parallel, meaning we use only the originally observed labels when computing the mode, not the predicted ones.

Looking at the graph, orange nodes have attribute value 1, blue nodes have value 0, and white nodes (unknown) are $\{H, D\}$. The known orange nodes are $\{B, C, F, G\}$. The known blue nodes are $\{A, E\}$. For each white node, we identify its neighbors and take the mode:

Node	Neighbors	Known Labels	Mode
H	$\{A, E, B\}$	$A = 0, B = 1, E = 0$	0
D	$\{B, C\}$	$B = 1, C = 1$	1

Thus, the local smoothing algorithm predicts:

$$x_H^* = 0, \quad x_D^* = 1$$

Thus, the two white nodes x_D, x_H are predicted to be: **orange** and **blue**, respectively.

4. Node2Vec Edge Prediction

In this exercise, you will use the Node2Vec algorithm that we saw in class to predict missing edges.

Imports

```
import igraph as ig                # graphing library
import matplotlib.pyplot as plt    # data viz
import pandas as pd                # dataframes
import numpy as np                 # numeric operations
import random                      # random sampling
```

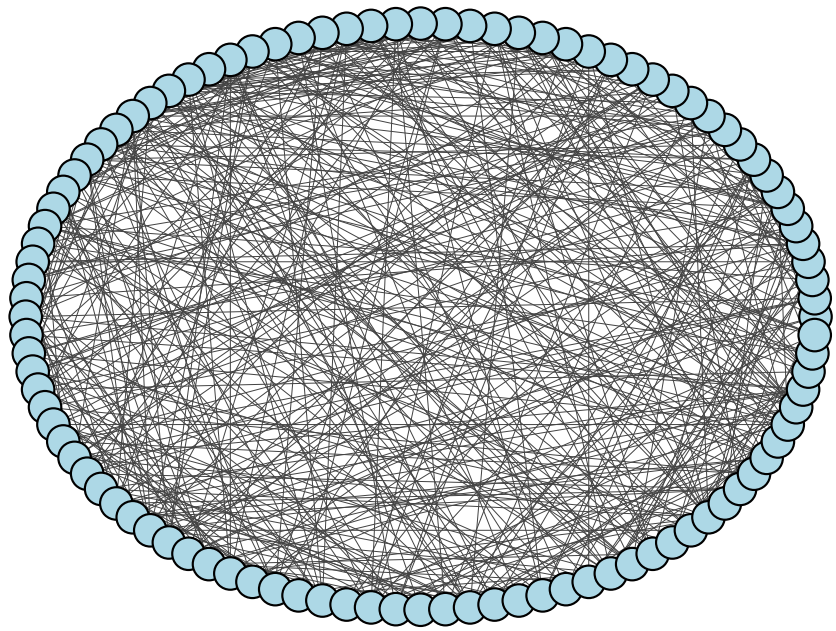
(a) Generate Erdos-Renyi graph

Generate an Erdos-Renyi random network with $n = 100$ and $\langle k \rangle = 10$.

```
random.seed(0) # set seed for reproducibility
# avg degree (c) equal to 10
n = 100
c = 10
m = int((n * c) / 2)
rg = ig.Graph.Erdos_Renyi(n=100, m=m,
                          directed=False,
                          loops=False)
ig.summary(rg)
```

```
IGRAPH U--- 100 500 --
```

```
# graph
fig, ax = plt.subplots(figsize = (8,6))
ig.plot(
    rg,
    target=ax,
    vertex_size=15,
    edge_width = .1,
    vertex_color = 'lightblue',
    layout='circle'
    )

plt.show()
```



(b) Remove edges

Remove a fixed percentage (20%) of the edges at random from the generated graph to simulate missing links.

i. How many edges are in the original graph, and how many will you remove?

There are 500 edges in the original graph; we will remove 100 (since we arbitrarily chose 20%). This is because the number of edges can be found as $m = \frac{n \cdot <k>}{2}$, where $c \equiv < k >$ is the 'average degree' of the network, meaning the average number of connections each node contains. Thus, with $n = 100$ nodes and an average degree of $c = 10$ we get that there are $m = 500$ edges. By removing 20%, we thereby end up with a graph of 400 edges.

```
# get list of edges from network
list_of_edges = rg.get_edgelist()
len(list_of_edges) # 500 edges
```

```
500
```

ii. What strategy will you use to randomly select the edges to be removed?

The chosen strategy involves calling up the 'random' library and generating a random sample, without replacement, of $m \cdot 0.20 = 100$ items in the range(m), with each draw being exactly 20% probability.

Afterwards, we using the built in igraph function 'x.delete_edges()', of which we feed in our list of random values generated, which correspond to the position in the edge list generated from the Erdos-Renyi graph, and this returns to us an edgelist of length 400.

iii. How will you store or mark these removed edges for later evaluation?

Before applying the 'x.delete_edges()' function to our Erdos-Renyi graph, which permanently remove the 100 edges, we can use the 'x.get_edges()' function to get a complete list of all our edges between nodes, and store it as an object. Afterwards, we can use simply iterate through the list and extract the elements that correspond to the values randomly generated from our 'random_edge_list', successfully storing the 100 edges before removing them.

```
# remove 20% edges randomly
random_edge_list = random.sample(population = list(range(m)),
                                 k = int(m*0.2))
len(random_edge_list) # 100 edges chosen to be removed
```

```
100
```

```
# store our deleted edges as a variable before applying
all_edges = rg.get_edgelist()
removed_edges_list = [all_edges[i] for i in random_edge_list]
```

```
# apply to our original erdos-renyi graph
rg.delete_edges(random_edge_list)
len(rg.get_edgelist()) # new graph has 400 edges
```

```
400
```

iv. What challenges might arise regarding the connectivity of the graph, and how would you address them?

By removing 20% of our connections in the graph, there is a non-zero chance that some nodes become isolated with a degree of 0, or perhaps there may arise a clear separation into multiple different components. This would lead to a failure of the Node2Vec algorithm, as it relies on the assumption of *random walks* through the network. With possible complete edge disconnections, this would likely perform poorly in terms of edge predictions.

A possible solution to this problem would to take multiple different random samples and apply them until we can ensure that the graph is fully connected still; we can check this easily using the 'x.is_connected()' function from igraph, which returns a boolean True/False if the graph is connected or not. Once we find a graph that returns "True," we can proceed without worries.

```
# test connectivity of graph
rg.is_connected()
```

```
True
```

(c) Node2Vec embeddings

```
import networkx as nx
import node2vec as n2v                                # node2vec algo
from sklearn.ensemble import RandomForestClassifier  # random forest algo
from sklearn.metrics import accuracy_score           # AUC/ROC scores
from sklearn.metrics.pairwise import cosine_similarity as cos_sim
from sklearn.decomposition import PCA
from sklearn.metrics import roc_curve, auc
```

i. What are the key parameters for your chosen embedding algorithm?

The key parameters for the `node2vec.Node2Vec()` algorithm include:

- **`dimensions`**: Size of the output embedding vector per node. Higher dimensions capture more structural nuance but increase memory and training time.
- **`walk_length`**: Number of nodes visited per random walk. Longer walks capture more global structure.
- **`num_walks`**: Number of random walks generated per node. More walks give better co-occurrence statistics for the skip-gram model.
- **`workers`**: Number of parallel CPU threads. Affects only runtime, not the output.
- `p` *(return parameter)*: Controls the probability of immediately backtracking to the previous node.
- `q` *(in-out parameter)*: Controls bias toward nodes farther from the previous node.

High `p` → more exploratory, DFS-like walks.

Low `q` → outward/DFS; high `q` → local/BFS. At `p=q=1` the algorithm reduces to DeepWalk.

```
# build a networkx graph from igraph object by copying edges
rg_nx = nx.Graph()
rg_nx.add_nodes_from([v.index for v in rg.vs])               # add all nodes first
rg_nx.add_edges_from([(e.source, e.target) for e in rg.es])  # then edges

# implement node2vec algo
node2vec_rg = n2v.Node2Vec(
    graph = rg_nx,      # graph object: MUST BE NX GRAPH!
    dimensions=64,      # embedding size
    walk_length=30,     # steps per walk
    num_walks=200,      # walks per node
    workers=4           # parallel threads
)
```

ii. How does each parameter influence the quality and efficiency of the embeddings?

- **`dimensions`**: Higher values improve representational capacity but slow training and may overfit on small graphs. 64 is a common default.
- **`walk_length`**: Longer walks expose the model to more distant neighborhoods, improving global structure capture, but add compute cost with diminishing returns.
- **`num_walks`**: More walks per node increase the reliability of co-occurrence counts fed to skip-gram. Critical for sparse graphs where any single walk may miss important neighbors.

- **workers**: More workers reduce wall-clock time linearly (up to CPU core count) with no effect on embedding quality.
- **p and q**: These are the most impactful quality parameters. They control the DFS/BFS trade-off: DFS-biased walks learn community membership; BFS-biased walks learn structural roles (e.g., hub vs. leaf). For link prediction, a mild DFS bias tends to work well.

iii. Can you visually represent the nodes in vector space?

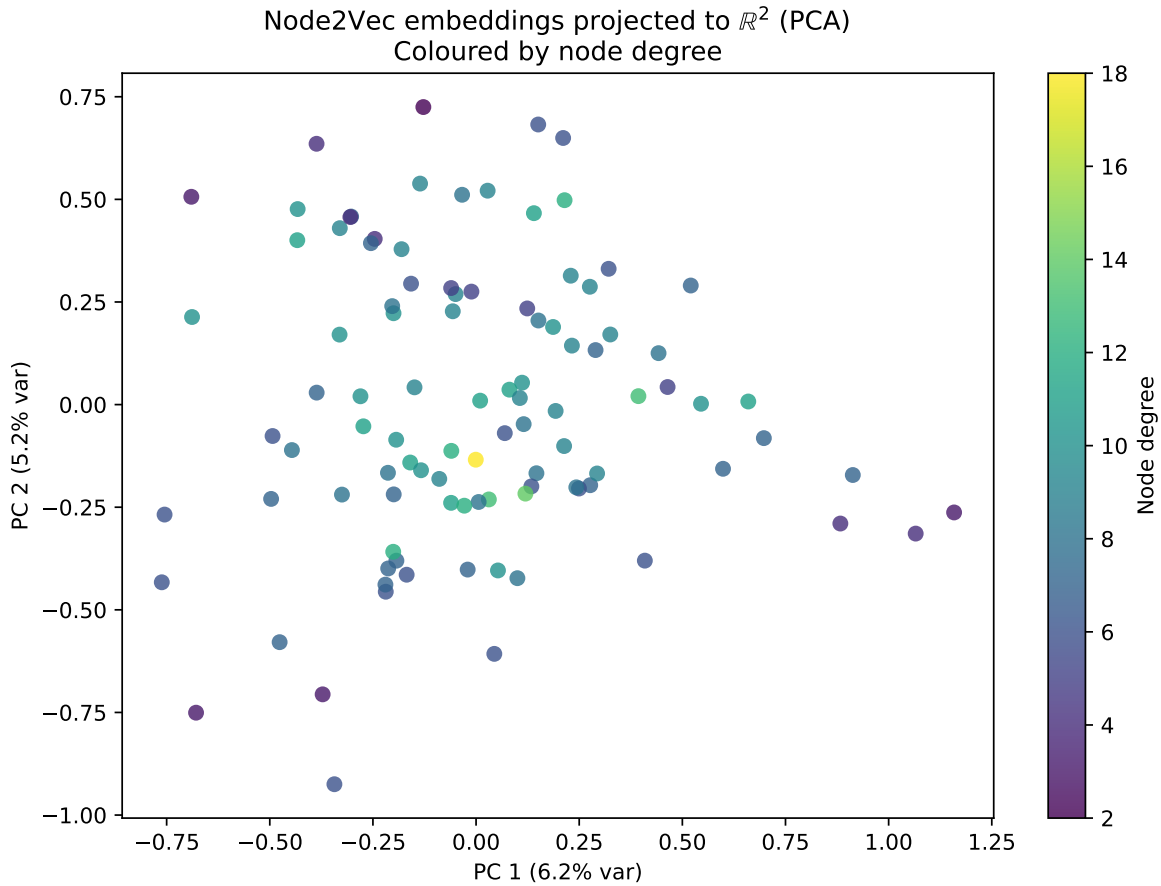
```
from sklearn.decomposition import PCA

# fit the skip-gram model on the walks generated above
model = node2vec_rg.fit(window=10, min_count=1, batch_words=4)

# extract the 64-dim embedding for every node
nodes      = list(rg_nx.nodes())
embeddings = np.array([model.wv[str(n)] for n in nodes])

# project to 2D with PCA so we can plot it
pca    = PCA(n_components=2, random_state=0)
emb_2d = pca.fit_transform(embeddings)
print(f"Variance explained by 2 PCs: {pca.explained_variance_ratio_.sum():.1%}")
```

```
Variance explained by 2 PCs: 11.4%
```

```
# compute node degree in the training graph (after edge removal) for colouring
degrees = np.array([rg_nx.degree(n) for n in nodes])

fig, ax = plt.subplots(figsize=(8, 6))
sc = ax.scatter(emb_2d[:, 0], emb_2d[:, 1],
                c=degrees, cmap='viridis', s=40, alpha=0.8)
plt.colorbar(sc, ax=ax, label='Node degree')
```

```
<matplotlib.colorbar.Colorbar object at 0x0000020D169C8D70>
```

```
ax.set_title("Node2Vec embeddings projected to $\\mathbb{R}^2$ (PCA)\nColoured by node degree")
ax.set_xlabel(f"PC 1 ({pca.explained_variance_ratio_[0]:.1%} var)")
ax.set_ylabel(f"PC 2 ({pca.explained_variance_ratio_[1]:.1%} var)")
plt.tight_layout()
plt.show()
```



(d) Cosine similarity scoring

For each missing edge, compute a similarity score using cosine similarity between their node embeddings.

i. How is cosine similarity defined mathematically?

Cosine similarity between two embedding vectors $\mathbf{e}_u$ and $\mathbf{e}_v$ is:

$$\cos(\mathbf{e}_u, \mathbf{e}_v) = \frac{\mathbf{e}_u \cdot \mathbf{e}_v}{\|\mathbf{e}_u\| \, \|\mathbf{e}_v\|}$$

It measures the cosine of the angle between the two vectors, ranging from -1 (opposite) to 1 (identical direction), ignoring differences in magnitude.

```
# Demonstrate the formula on an actual edge from rg_nx
u, v = removed_edges_list[0]
eu = model.wv[str(u)]
ev = model.wv[str(v)]

dot_product = np.dot(eu, ev)
norm_u      = np.linalg.norm(eu)
norm_v      = np.linalg.norm(ev)
```

```
cosine_manual = dot_product / (norm_u * norm_v)

print(f"Edge ({u}, {v}):")
```

```
Edge (49, 58):
```

```
print(f"  dot product          = {dot_product:.4f}")
```

```
  dot product          = 0.5741
```

```
print(f"  ||e_u|| * ||e_v||    = {norm_u:.4f} * {norm_v:.4f} = {norm_u*norm_v:.4f}")
```

```
  ||e_u|| * ||e_v||    = 1.6765 * 1.5299 = 2.5650
```

```
print(f"  cosine similarity    = {cosine_manual:.4f}")
```

```
  cosine similarity    = 0.2238
```

```
from sklearn.metrics.pairwise import cosine_similarity as cos_sim

# positive examples: the 100 removed edges
pos_scores = [
    cos_sim(model.wv[str(u)].reshape(1,-1),
            model.wv[str(v)].reshape(1,-1))[0,0]
    for u, v in removed_edges_list
]

# negative examples: random non-edges of equal size
removed_set = set(map(frozenset, removed_edges_list))
rng = np.random.default_rng(42)
all_nodes_arr = list(rg_nx.nodes())
neg_scores = []
while len(neg_scores) < len(pos_scores):
    a, b = rng.choice(all_nodes_arr, size=2, replace=False)
    a, b = int(a), int(b)
    if not rg_nx.has_edge(a, b) and frozenset([a, b]) not in removed_set:
        neg_scores.append(
            cos_sim(model.wv[str(a)].reshape(1,-1),
                    model.wv[str(b)].reshape(1,-1))[0,0]
        )
```

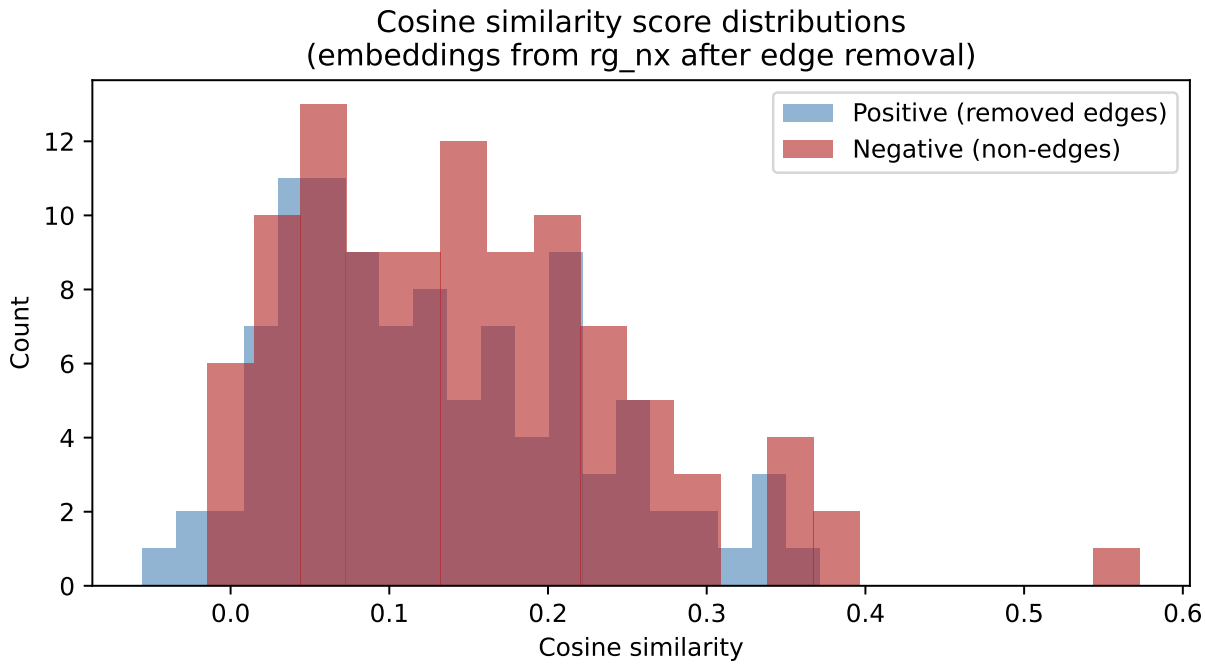
```
print(f"Positive (removed edges) - mean: {np.mean(pos_scores):.3f}, std: {np.std(pos_scores):.3f}")
```

```
Positive (removed edges) - mean: 0.129, std: 0.096
```

```
print(f"Negative (non-edges)     - mean: {np.mean(neg_scores):.3f}, std: {np.std(neg_scores):.3f}")
```

```
Negative (non-edges)     - mean: 0.150, std: 0.104
```

```
# visualise the score distributions
fig, ax = plt.subplots(figsize=(7, 4))
ax.hist(pos_scores, bins=20, alpha=0.6, label='Positive (removed edges)', color='steelblue')
ax.hist(neg_scores, bins=20, alpha=0.6, label='Negative (non-edges)',     color='firebrick')
ax.set_xlabel('Cosine similarity')
ax.set_ylabel('Count')
ax.set_title('Cosine similarity score distributions\n(embeddings from rg_nx after edge removal)')
ax.legend()
plt.tight_layout()
plt.show()
```



ii. Why might cosine similarity be an appropriate measure for scoring missing edges?

Node2Vec encodes structural proximity as the *direction* of a vector, not its magnitude. I.e., nodes that frequently co-occur in random walks end up pointing in similar directions in embedding space. Cosine similarity captures that directional alignment, making it a natural scoring function. Two nodes with similar neighborhoods will have a high cosine score regardless of the size of their own individual embedding vectors.

```
# Compare cosine similarity (scalar) vs Hadamard product (vector) for the same edge
u, v = removed_edges_list[0]
eu = model.wv[str(u)]
ev = model.wv[str(v)]

hadamard  = eu * ev                                          # element-wise product -> 64-dim vector
cosine    = cos_sim(eu.reshape(1,-1), ev.reshape(1,-1))[0,0]  # single scalar

print(f"Edge ({u}, {v}):")
```

```
Edge (49, 58):
```

```
print(f"  Cosine similarity (scalar)          : {cosine:.4f}")
```

```
  Cosine similarity (scalar)          : 0.2238
```

```
print(f"  Hadamard product (first 8): {hadamard[:8].round(4)}")
```

```
  Hadamard product (first 8): [ 0.1414 -0.0322 -0.0014 -0.0065 -0.0234  0.0093  0.0002  0.0002]
```

```
print(f"\nCosine: one number to threshold directly.")
```

```
Cosine: one number to threshold directly.
```

```
print(f"Hadamard: {len(hadamard)}-dim feature vector; needs a classifier (e.g. logistic regression).")
```

```
Hadamard: 64-dim feature vector; needs a classifier (e.g. logistic regression).
```

iii. Discuss any potential benefits or drawbacks of using cosine similarity compared to other operators

Cosine similarity collapses the pair comparison to a single scalar, which is simple and requires no additional training. The downside is that it discards per-dimension information and assumes symmetry.

Hadamard product ($\mathbf{e}_u \cdot \mathbf{e}_v$) produces a full feature vector for each edge, which can then be passed to a logistic regression or gradient boosted model. This can learn which embedding dimensions are predictive of edges and handle asymmetric relationships well. The tradeoff is that it requires labelled training data and a supervised classifier, which can add complexity to the pipeline.

(e) ROC curve and AUC

Combine the cosine similarity scores from both the positive examples and the negative examples. Construct an ROC curve and compute the Area Under the ROC Curve (AUC).

i. What does the ROC curve represent in the context of link prediction?

The ROC curve plots the *true positive rate* (fraction of real missing edges correctly scored above threshold) against the *false positive rate* (fraction of non-edges incorrectly scored above threshold) as the classification threshold is swept from high to low. Each point on the curve corresponds to one threshold choice. In link prediction, it shows how well the cosine score separates truly missing edges from node pairs that never connected.

ii. How do you interpret the AUC value with respect to the quality of your predictions?

The AUC (Area Under the ROC Curve) is the probability that the model assigns a higher cosine score to a randomly chosen positive pair (removed edge) than to a randomly chosen negative pair (non-edge). An AUC of 1.0 indicates perfect discrimination; 0.5 indicates no better than random. For our ER graph, we find a AUC value of ~0.525, which is expected for a random probability graph.

iii. What does an AUC value of 0.5 versus a value close to 1.0 indicate about your model's performance?

An AUC = 0.5 indicates that the model's scores are indistinguishable from random. In other words, the cosine similarity provides no useful signal for predicting which pairs are edges.

For an AUC close to 1.0, this indicates that the model almost perfectly ranks true missing edges above non-edges, meaning the embeddings have captured the structural patterns that determine edge formation.

For an Erdos-Renyi graph, where edges generate uniformly at random with no community structure, we expect an AUC close to 0.5 since node2vec embeddings have no meaningful pattern to learn from.

```
from sklearn.metrics import roc_curve, auc

# build labels and scores from pos_scores / neg_scores computed above
y_true  = np.array([1]*len(pos_scores) + [0]*len(neg_scores))
y_score = np.array(pos_scores + neg_scores)

fpr, tpr, _ = roc_curve(y_true, y_score)
roc_auc = auc(fpr, tpr)

fig, ax = plt.subplots(figsize=(6, 5))
ax.plot(fpr, tpr, label=f'Node2Vec + cosine (AUC = {roc_auc:.3f})')
```

```
[<matplotlib.lines.Line2D object at 0x0000020D1BDB51D0>]
```

```
ax.plot([0,1], [0,1], 'k--', label='Random (AUC = 0.5)')
```

```
[<matplotlib.lines.Line2D object at 0x0000020D1BDB5310>]
```

```
ax.set_xlabel('False Positive Rate')
```

```
Text(0.5, 0, 'False Positive Rate')
```

```
ax.set_ylabel('True Positive Rate')
```

```
Text(0, 0.5, 'True Positive Rate')
```

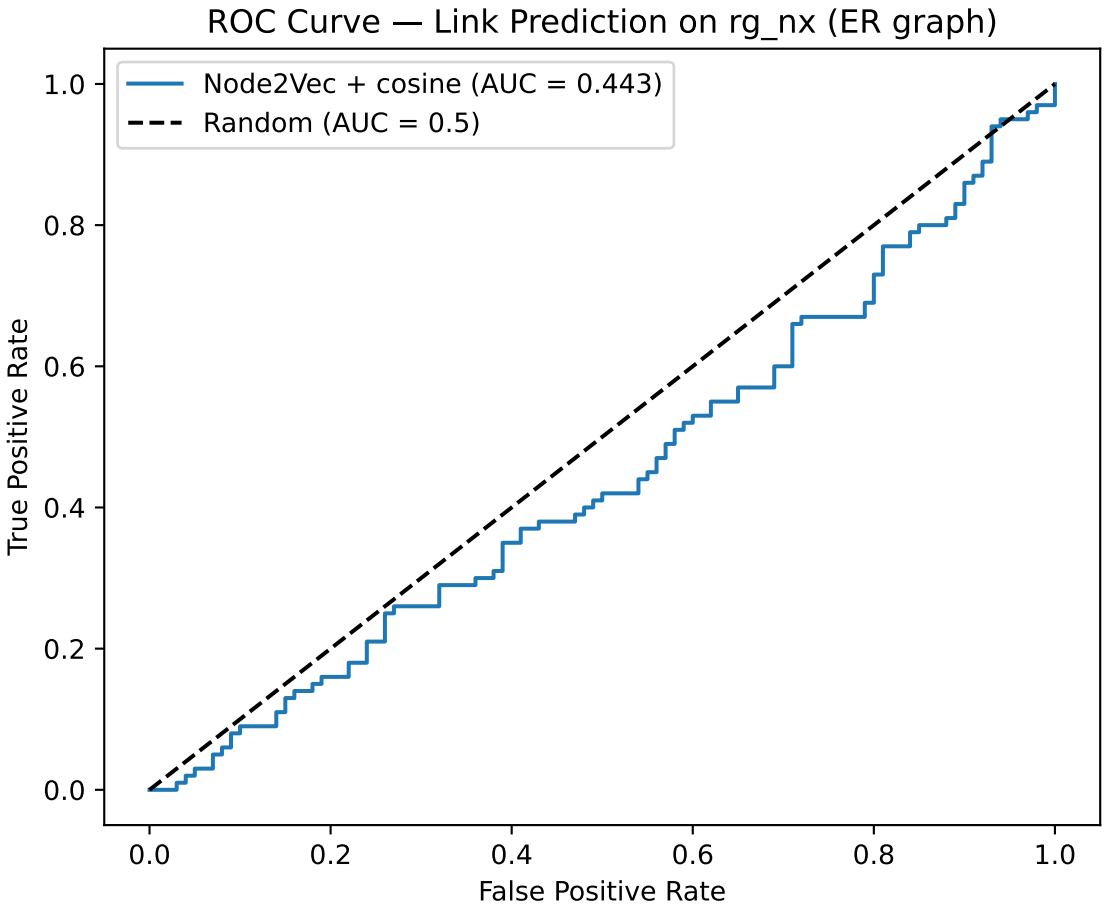
```
ax.set_title('ROC Curve - Link Prediction on rg_nx (ER graph)')
```

```
Text(0.5, 1.0, 'ROC Curve - Link Prediction on rg_nx (ER graph)')
```

```
ax.legend()
```

```
<matplotlib.legend.Legend object at 0x0000020D1BDB5450>
```

```
plt.tight_layout()
plt.show()
```




```
print(f"AUC = {roc_auc:.4f}  ({'near random, expected for ER graph' if roc_auc < 0.6 else 'above random
```

```
AUC = 0.4432  (near random, expected for ER graph)
```

(f) Analysis and discussion

i. How effective was cosine similarity as a scoring method for predicting missing links?

On an Erdos-Renyi graph, cosine similarity is expected to perform poorly; the AUC should be near 0.5. This is not a failure of the method but rather of the data: ER graphs have no community structure, so nodes connected by an edge are structurally indistinguishable from unconnected pairs. Node2Vec learns embeddings that reflect co-occurrences from random walks; however, with a uniform edge formation there is no pattern to capture. Cosine similarity (and any embedding-based method) works best on graphs with strong communities or degree heterogeneity.

ii. How might different embedding parameters or alternative scoring methods impact the results?

On a structured graph, increasing `dimensions` and `num_walks` generally improves AUC. The `p` and `q` parameters matter most: DFS-biased walks (`q < 1`) tend to produce better community-level embeddings for link prediction, while BFS-biased walks (`q > 1`) capture local structural roles. On an ER graph, tuning these

parameters has little effect since there is no underlying structure to exploit. Replacing cosine similarity with Hadamard + logistic regression would also increase AUC on structured graphs by learning which embedding dimensions are predictive, but would not help here for the same reason.

```
import node2vec as n2v

def run_link_pred(rg_nx, removed_edges_list, removed_set, dims, walk_len, nwalks):
    """Train Node2Vec with given params and return the AUC on rg_nx."""
    m = n2v.Node2Vec(rg_nx, dimensions=dims, walk_length=walk_len,
                     num_walks=nwalks, workers=1, quiet=True)
    wv = m.fit(window=10, min_count=1, batch_words=4).wv

    p = [cos_sim(wv[str(u)].reshape(1,-1), wv[str(v)].reshape(1,-1))[0,0]
         for u, v in removed_edges_list]

    rng2 = np.random.default_rng(0)
    nn = list(rg_nx.nodes())
    n_ = []
    while len(n_) < len(p):
        a, b = int(rng2.choice(nn)), int(rng2.choice(nn))
        if a != b and not rg_nx.has_edge(a,b) and frozenset([a,b]) not in removed_set:
            n_.append(cos_sim(wv[str(a)].reshape(1,-1), wv[str(b)].reshape(1,-1))[0,0])

    yt = np.array([1]*len(p)+[0]*len(n_))
    ys = np.array(p + n_)
    return auc(*roc_curve(yt, ys)[:2])

# compare dims: 16, 32, 64, 128  (walk_length=30, num_walks=50 for speed)
print("dimensions | AUC")
```

```
dimensions | AUC
```

```
print("-----------+------")
```

```
-----------+------
```

```
for d in [16, 32, 64, 128]:
    a = run_link_pred(rg_nx, removed_edges_list, removed_set, dims=d, walk_len=30, nwalks=50)
    print(f"    {d:>3}    | {a:.3f}")
```

```
     16    | 0.439
     32    | 0.458
     64    | 0.463
    128    | 0.441
```

```
# compare walk lengths: 10, 20, 30  (dims=64, num_walks=50)
print("\nwalk_length | AUC")
```

```
walk_length | AUC
```

```
print("------------+------")
```

```
------------+------

for wl in [10, 20, 30]:
    a = run_link_pred(rg_nx, removed_edges_list, removed_set, dims=64, walk_len=wl, nwalks=50)
    print(f"    {wl:>2}      | {a:.3f}")

    10      | 0.420
    20      | 0.438
    30      | 0.464
```

iii. What are the potential limitations of your current methodology, and what improvements would you suggest?

Limitations: - The Erdos-Renyi graph is a poor test case, as its lack of community structure from random edge generation makes any embedding-based method essentially unlearnable. - Negative sampling is random and does not account for pairs that are structurally similar but disconnected. - We chose an arbitrary 20% edge removal rate; with different chosen values, results may vary significantly, especially if removing edges past a certain threshold disconnects portions of the graph and/or leads to isolated nodes.

Improvements: - Use a graph with known community structure (e.g. karate club) to demonstrate the method's potential. - Replace random sampling with degree-stratified sampling to avoid disconnections between low-degree nodes. - Use Hadamard features + logistic regression instead of raw cosine similarity for a stronger baseline.